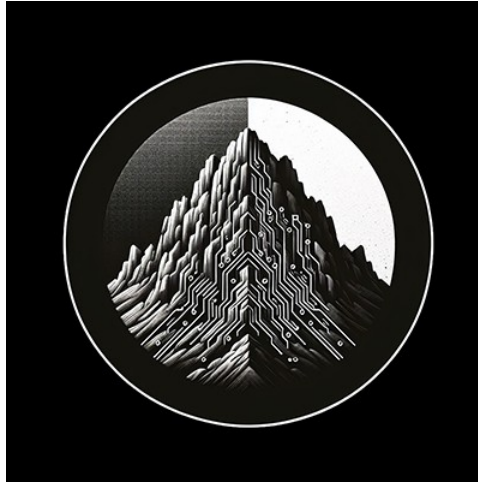




RTL Design Sherpa

STREAM Characterization DMA Performance Report

June 2026

Table of Contents

1 Readme.....	4
2 perf/ — STREAM DMA characterization runs.....	4
2.1 Files.....	4
2.1.1 matrix_2026-06-15 — all-timing-fixes bitstream.....	4
2.2 Picking the right efficiency number.....	5
2.3 Columns (current runner).....	6
2.4 Methodology notes.....	7
2.5 Reproduce.....	7

List of Tables

No tables in this document.

1 Readme

Generated: 2026-06-15

2 perf/ — STREAM DMA characterization runs

CSV exports of FPGA characterization measurements. Methodology: see `projects/NexysA7/stream_characterization/DMA_UTILIZATION_MEASUREMENT.md`.

2.1 Files

File	Source	Rows	Notes
matrix_2026-06-15.csv / .json	full default matrix	40	current — all-timing-fixes bitstream (see below)
char_matrix_2026-06-05.csv	full default matrix	40	desc x ch sweep at 1MB/desc (pre-cleanup, see note)
respdelay_sweep_2026-06-05.csv	resp-delay axis	32	4 configs x 8 delays (0..64 cyc)

2.1.1 matrix_2026-06-15 — all-timing-fixes bitstream

Captured on the bitstream that closes timing with **WNS +0.048 ns** at 100 MHz (xc7a100t), carrying the full set of timing/feature work: both CAM pipelines (monbus_cam_pipe route-bound match + monitor TRANS_CAM_PIPELINE with gated MAX-3 block margin), half-beat packing, monbus compression, the wrapping trace write pointer, and the pblock_monbus floorplan + timing-directive build.

Result: all 40 configs pass, datapath/bus end-to-end utilization sits in a **94.06–94.12 %** band across the entire desc×ch matrix (engine ceiling, flat regardless of channel count or descriptor depth), and net bus throughput is **1435–1436 MB/s** against the 1526 MB/s one-direction ceiling. The timing fixes change none of the functional throughput — they make the bitstream valid (positive slack) without regressing the 94.1 % engine efficiency measured on earlier builds.

The `trace.overflow` flag trips on the eight `16desc_*` configs because the on-chip debug trace SRAM is only 2048 beats deep (16 desc × 128 beats > 2048). This is benign: it bounds only the captured waveform trace, not the `axi_bus_meter` counters or the timer window, so every efficiency number above is unaffected.

Note on the 2026-06-05 CSVs. These pre-date the runner cleanup documented below. Their `throughput_MBps`, `E2E_util_pct`, and

overhead_delta_pp columns are **host-wall-clock measurements** that were polluted by the runner's 50 ms poll-loop sleep and a wrong-by-2x bus-ceiling denominator. Read those three columns as host-observed end-to-end metrics, **not** as bus efficiency. The bus efficiency numbers in those files are datapath_{R,W,E2E}_pct (steady-state, burst-only window from the on-chip PMU), which are clean: 94.1% across every config and every resp-delay setting from 0 to 64 cycles. Newly captured CSVs (post-fix) should use the bus-side columns documented in the next section instead.

2.2 Picking the right efficiency number

The runner produces three independent throughput-ish numbers per config, and they answer different questions. Pick the one that matches the question being asked:

Metric	Window	What it answers	Use it for
datapath_E2E_pct	first→last productive beat (on-chip PMU)	“When the engine is actively bursting, what fraction of cycles are productive?”	Engine capability ceiling — independent of descriptor pacing and host overhead.
bus_e2e_util_pct	full FPGA timer window (kick → done)	“Of every ackl cycle from kick to done, what fraction was a productive beat?”	True bus utilization for the workload (includes inter-descriptor gaps).
throughput_MBps (wall)	host wall clock, time.time() around kick→poll	“How long did the whole DMA take from the host's perspective?”	End-to-end latency including UART poll cadence — not a bus metric.

Quoting datapath_E2E_pct as “the engine's efficiency” is the defensible headline. Quoting bus_e2e_util_pct is the defensible “workload as configured” number. Quoting throughput / theoretical_max where throughput is wall-clock and theoretical_max is single-bus peak is the wrong answer to almost every question.

2.3 Columns (current runner)

Run-identification:

- `date, time`: wall clock of the run
- `config, channels, descriptors, desc_KB`: workload axes
- `mb_moved`: net bytes the DMA moved end-to-end (R then W, so the on-bus traffic is 2× this).
- `resp_delay_cycles` (sweep only): per-beat R+B hold injected by `axi_response_delay`.

Bus-side (sourced from the on-chip harness timer + `axi_bus_meter` counters — no UART overhead, no wall-clock contamination):

- `bus_time_s`: `cycles_total / aclk_hz`. Span from descriptor kick to write-side done. The exact FPGA-side DMA duration.
- `bus_throughput_MBps`: `mb_moved / bus_time_s`. Net-bytes-moved divided by FPGA time. Compare to `bus_max_net_moved_MBps`.
- `bus_max_one_dir_MBps`: theoretical ceiling for one direction — `aclk_hz * DATA_WIDTH_BYTES`. At 100 MHz + 128b that's 1600 MB/s.
- `bus_max_net_moved_MBps`: theoretical ceiling for net bytes moved end-to-end. Half of `bus_max_one_dir_MBps` because the DMA reads *and* writes each byte; both directions contend for the same engine budget. ~800 MB/s at the default operating point.
- `bus_e2e_util_pct`: `productive_beats / cycles_total`. Of every aclk in the FPGA timer window, what fraction was a productive beat. This is the bus efficiency number that's robust against UART, poll cadence, and the wrong-denominator trap.
- `datapath_{R,W,E2E}_pct`: harness-reported steady-state productive% inside the first→last beat window per side (methodology def 2.1). Ignores any inter-descriptor gaps and warm-up cost. Headline metric for “what is the engine capable of when it's running.”
- `{R,W}_{prod,bp,starv}_pct`: four-bucket breakdown of the data-phase window (methodology Section 3). `prod + bp + starv + idle = 100%`.

Host-side (kept for transparency; do not use as efficiency metrics):

- `dma_time_s`: `time.time() - t0` across kick → poll-completion. Includes every UART CSR round-trip and the host's poll sleep.

- `throughput_MBps (wall): mb_moved / dma_time_s`. Asymptotes upward as transfer size grows because the per-DMA host overhead amortizes — that’s a property of the host, not the engine.

2.4 Methodology notes

- The on-chip timer counts `aclk` cycles between descriptor kick and the moment done latches (write-side beat count reaches `TIMER_EXPECTED_BEATS`). The bus meters freeze at the same done signal, so the bucket sums equal the timer window exactly. That’s why dividing productive by either `cycles_total` (E2E) or the bucket sum (datapath) is consistent — the same window is being divided differently.
- `dma_time_s` was contaminated by a 50 ms host poll sleep until the runner cleanup; that’s now 1 ms by default and reconfigurable via `--poll-interval-ms`. Even at 1 ms, prefer `bus_time_s` for any efficiency math.
- For non-100 MHz bitstreams, pass `--aclk-mhz` so `bus_time_s` and the bus throughput ceiling are computed against the right clock.

2.5 Reproduce

```
cd flows-stream-bridge/host
```

```
# Full 40-config matrix (default 1ms poll, 100 MHz aclk):
python3 run_characterization.py --port /dev/ttyUSB2 --output
matrix.json
```

```
# resp-delay axis (4 configs x 8 delays = 32 runs):
python3 run_characterization.py --port /dev/ttyUSB2 \
  --configs 1desc_1ch_1MB 16desc_1ch_1MB 16desc_4ch_1MB
16desc_8ch_1MB \
  --resp-delays 0,1,2,4,8,16,32,64 \
  --output respdelay.json
```

```
# Override host poll cadence / clock (e.g. for a 200 MHz build):
python3 run_characterization.py --port /dev/ttyUSB2 \
  --poll-interval-ms 0.5 --aclk-mhz 200 --output fast.json
```

The runner emits JSON; convert it to the column set above with:

```
cd flows-stream-bridge/host
python3 perf_json_to_csv.py ../../reports/perf/matrix.json \
  --out ../../reports/perf/matrix.csv --date 2026-06-15 --time
13:58:32
```

`perf_json_to_csv.py` maps one CSV row per result dict (bus-side columns first, host-side last). The `--date/--time` flags only stamp the run-identification columns; all measurements come from the JSON.